Accelerating Static Timing Analysis Using CPU–GPU Heterogeneous Parallelism

Zizheng Guo, Tsung-Wei Huang, *Member, IEEE*, and Yibo Lin, *Member, IEEE*

***Abstract*—Static timing analysis (STA) is an essential yet time-consuming task during the circuit design flow to ensure the correctness and performance of the design. Thanks to the advancement of general-purpose computing on graphics processing units (GPUs), new possibilities and challenges have arisen for boosting the performance of STA. In this work, we present an efficient and holistic GPU-accelerated STA engine. We accelerate major STA tasks, including levelization, delay computation, graph propagation, and multicorner analysis, by developing high-performance GPU kernels and data structures. By dividing the STA workloads into CPU–GPU concurrent tasks with managed dependencies, our acceleration framework supports versatile incremental updates. Furthermore, we have extended our approach to multicorner analysis by exploring a large amount of corner-level data parallelism using GPU computing. Our implementation based on the open-source STA engine OpenTimer has achieved up to 4.07× speed-up on single corner analysis, and up to 25.67× speed-up on multicorner analysis on TAU 2015 contest designs and a 14-nm technology.**

***Index Terms*—Heterogeneous parallelism, static timing analysis (STA).**

I. Introduction

With the advancement of design complexities and process technology, the overall circuit design closure is increasingly bounded by the timing analysis on circuit graphs consisting of hundreds of process corners and billions of transistors. To ensure the timing correctness and performance of the design, static timing analysis (STA) is frequently invoked in the iterative and incremental updates inside optimization algorithms [1]. In response to millions of design modifications performed by the optimization flow, the timer is required to provide instant and accurate feedback on slack values and timing criticality changes. To achieve acceptable performance and design turnaround time, it is crucial to have an efficient STA engine.

A number of parallel STA algorithms have been proposed in previous works, including both commercial tools and academic research [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]. While each has their advantages and drawbacks, nearly all of these algorithms are inherently bound by the *multithreaded* parallelism on a platform with central processing units (CPUs) and multicore architecture. While some of these attempts have gained runtime benefits, most of them stop scaling beyond 8–16 CPU cores [11], without more scalable replacements above that. With the increasing computing capacity of modern graphics processing units (GPUs), new possibilities have arisen for boosting the performance of STA engines. However, it is extremely challenging to develop an efficient STA engine running on a CPU–GPU heterogeneous platform. Computing the signal timing across a circuit graph involves both diverse computational patterns and irregular memory access, including dynamic data structures, graph-oriented computing, recursion, and branch-and-bound, to name a few [3]. These patterns lead to a vast and complex STA workload consisting of nontrivial functional dependencies. We need very strategic data structure models and decomposition algorithms to obtain a reasonable STA runtime speed-up.

As design shifts to nanoscale, the timing effects from process, voltage, and temperature (PVT) changes are more and more tied to the successful tapeout of chips. The analysis of such effects is done by *multicorner* STA engines that address all combinations of PVT corners in independent STA runs, which bear a large memory footprint and huge runtime. Most previous research on multicorner STA acceleration [12], [13], [14], [15], [16], [17] is limited to CPU-based parallelism either within one or multiple STA processes. This organization cannot efficiently explore large data parallelism exhibited by multicorner settings.

In this work, we present a new STA implementation on a CPU–GPU heterogeneous platform. We propose GPU-efficient acceleration kernels and data structures to offload major STA computing steps to GPU. We implement our algorithms based on the open-source STA engine OpenTimer, designed by Huang et al. [3]. Our algorithm's core design philosophy is universally applicable and can be applied to other STA frameworks. The major contributions of this article are summarized in the following.

1) We divide the STA workloads into CPU–GPU concurrent tasks with managed dependencies by

Manuscript received 13 December 2022; revised 14 March 2023; accepted 6 June 2023. Date of publication 14 June 2023; date of current version 22 November 2023. This work was supported in part by the National Science Foundation of China under Grant 62034007 and Grant 62004006; in part by the National Science Foundation of U.S. under Grant CCF-2126672, Grant CCF-2144523 (CAREER), Grant OAC-2209957, and Grant TI-2229304; and in part by the 111 Project under Grant B18001. The preliminary version, titled "GPU-Accelerated Static Timing Analysis," has been presented at the International Conference on Computer-Aided Design (ICCAD) in 2020 [DOI: 10.1145/3400302.3415631]. This article was recommended by Associate Editor C.-K. Cheng. *(Corresponding author: Yibo Lin.)*

Zizheng Guo is with the School of Integrated Circuits, Peking University, Beijing 100871, China.

Tsung-Wei Huang is with the Department of Electrical and Computer Engineering, University of Wisconsin at Madison, Madison, WI 53706 USA.

Yibo Lin is with the School of Integrated Circuits, Peking University, Beijing 100871, China, also with the Institute of Electronic Design Automation, Peking University, Wuxi 214125, China, and also with the Beijing Advanced Innovation Center for Integrated Circuits, Beijing 100871, China (e-mail: yibolin@pku.edu.cn).

Digital Object Identifier 10.1109/TCAD.2023.3286261

1937-4151 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

leveraging task-based parallelism, effectively hiding data processing and memory latency.

2) We develop high-performance GPU kernels and data structures tailored to GPU parallelism for all major STA operations, including delay calculation, levelization, and graph propagation.
3) We implement our GPU-accelerated STA algorithms based on a real-world STA framework with support to industrial design formats and incremental timing. Our techniques provide valuable insight into CPU–GPU performance tradeoff in realistic scenarios.
4) We extend our GPU algorithms to multicorner timing analysis by exploring data parallelism across the corner dimension and proposing efficient memory mapping under GPU memory constraints. We have demonstrated a substantial extra performance benefit.

We evaluate our algorithm on the gate-level circuit netlists from TAU 2015 Timing Analysis Contest benchmarks consisting of large industrial designs [18]. We use a 14-nm technology to provide realistic single-corner and multicorner cell libraries. We have achieved a significant speed-up on both single-corner and multicorner STA. As an example, on two large circuit designs, netcard and leon2, we accelerate OpenTimer by 4.05× and 4.07× using one GPU for single-corner analysis. By computing 16 corners in parallel, we achieved another 5.66× and 6.34× speed-up on these designs. We also investigated the impact of several factors on incremental timing performance, such as gate number, net count, and incremental graph size, and provided recommendations on when to use GPU or CPU. Given the composite of software tradeoffs and architectural considerations we have made, we believe our STA algorithm delivers a novel acceleration methodology.

The organization of this article is listed as follows. In Section II, we introduce the STA background, GPU architecture, and our problem formulation. In Section III, we present details of our multicorner STA algorithm on GPU. In Section IV, we demonstrate the experimental results on STA runtime improvement. Section V summarizes this article.

II. Static Timing Analysis

In STA, circuits are represented as directed acyclic graphs (DAGs), where nodes represent pins of circuit components and edges represent pin interconnects. Fig. 1 illustrates an example STA graph consisting of four logic cells, five primary ports, and seven nets. In order to compute the signal arrival times (ATs) on this timing graph, a *graph-based* STA engine performs two major steps called *forward* and *backward propagation*. In forward propagation, timing quantities such as delay, slew, and RC are computed, and AT is accumulated according to data dependencies. In backward propagation, timing constraints and slack statistics such as required AT (RAT) are computed based on the forward propagation result. To keep our discussion focused, we compute cell delays through the nonlinear delay model (NLDM) based on 2-D lookup tables (LUTs) with load and slew indices, and net delays through the widely known Elmore delay model. These models are used

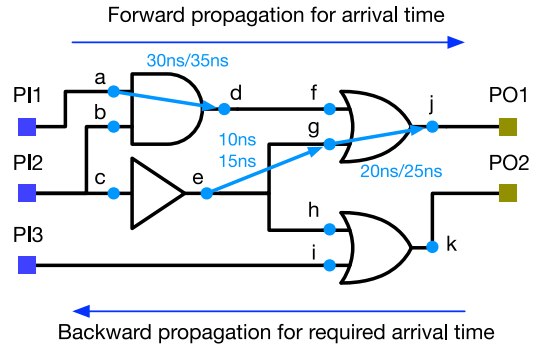


Fig. 1. STA graph of a circuit design. The blue nodes indicate pins of components like gates and I/O ports. The arrows indicate pin-to-pin connections. Delay values are quantified using best (min) and worst (max) scenarios.

in recent TAU 2014–2019 timing analysis contests [18], [19], [20] with golden results available. The node criticality is quantized by its *slack* defined as the difference between its AT and RAT. Setup check and hold check, respectively, refer to the late and early slack values.

A. Parallel STA Engines

The design complexity of modern VLSI systems is ever-increasing, with millions to even billions of pins and interconnect. Such a large amount of computation puts unprecedented pressure on the analysis speed of STA engines used to evaluate large designs. To shorten STA's substantial runtime, ongoing research has looked toward parallel and distributed STA frameworks [2], [3], [6], [7]. Huang et al. [3], [5], [6], for example, developed OpenTimer, a timing analysis engine that represents timing propagation jobs and their precedence using a dependency graph of tasks. Their result has demonstrated up to 2× runtime improvement over loop-based parallelization using OpenMP. Another attempt on parallel timing graph propagation for FPGA designs is performed by Murray et al. and demonstrated 9× speed-up using 32 CPU cores. Besides, new challenges on timing macromodeling [21], [22], common path pessimism removal [8], [9], [10], and incremental timing are also raised by recent TAU contests [18], [19], [20].

The multithreading performance based on CPUs generally saturates at roughly 8–16 threads, due to irregular computational patterns and threading overhead of STA [3], [7]. To overcome the scalability challenge, timing analysis with GPU acceleration is further investigated [7], [23]. Wang et al. [23] have explored GPU acceleration of the LUT interpolation step during cell delay computation while leaving other STA steps like levelization and net delay computation on CPU. Regarding the kernel runtime, a more than 10× speed-up has been demonstrated compared to their CPU version. The work by Murray and Betz [7] mentioned before also investigated timing propagation on GPUs. While their kernel runtime has been 6.2× faster, the overall propagation runtime has become even 0.9× slower over CPU, due to the memory transfer overhead not accounted for in kernel runtime. In addition to the above works, acceleration of statistical STA (SSTA) is also

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

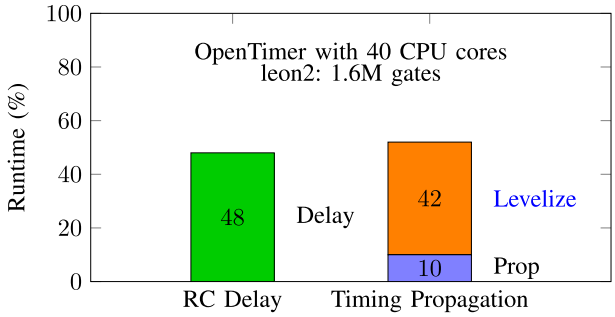


Fig. 2. OpenTimer runtime decomposition (40 CPU cores) inside one full timing process on a million-sized design.

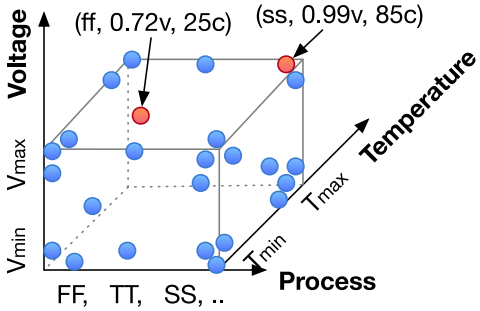


Fig. 3. PVT corners are combinations of PVT parameters. They can be visualized as points in a 3-D cube.

attempted using GPUs and FPGAs, which is a different scope of discussion [24], [25], [26].

To accurately address the runtime bottleneck of a real STA run, we have profiled the widely adopted open-source STA engine OpenTimer [3], [5]. As OpenTimer is reported to outperform commercial STA tools [3] in terms of speed, we regard its runtime footprint as a common scenario in high-performance STA engines. We draw its runtime decomposition for a full graph-based timing analysis in Fig. 2. As we can observe in the figure, the RC timing step, including RC trees construction and slew parameter updates across nets, takes up a large percentage (48%) of the runtime due to the vast amount of SPEF data to process [18]. Another significant portion of runtime (42%) is taken by constructing levelized task dependency graphs with a size proportional to the circuit size, on which timing propagation is conducted. Handling the signal relations between pins is a common burden in all STA engines, due to its difficulty to be parallelized.

B. Multicorner Timing Analysis in Advanced Nodes

With the continued increase of design complexity and advancements in technology nodes, the timing behavior of a circuit design is more and more involved with variations introduced across design process, manufacturing noise, and operating conditions (Fig. 3). Therefore, *multicorner* timing analysis is proposed by repeatedly applying the STA engine on different cell libraries characterized under different PVT conditions. The number of such combinations is often tens or hundreds to cover a large enough set of scenarios during sign-off, effectively magnifying the STA runtime by 10× or even 100×.

To reduce the runtime of the expensive multicorner analysis, researchers have proposed a number of approaches recently [12], [13], [14], [15], [16], [17]. One direction is to apply branch-and-bound during corner parameter selection and STA. For example, [17] proposes to prune the search space of multicorner exploration with delay upper-bound estimations. This is further improved by [14] which runs different branch-and-bound algorithms in parallel by CPU-based multithreading. Another direction is to approximate the corners by incorporating prior knowledge like the clock tree update in hold analysis [13] or the local linearity between similar corners and modes [15], [16]. One recent direction applies machine learning (ML) to multicorner STA by predicting unobserved corners given observed ones [12]. However, they still suffer from about 10% maximum error due to the inherent stability and explainability problem of ML models.

Despite extensive research on multicorner analysis, most of them are limited to CPU-based parallelism. Worse still, they introduce accuracy loss in the multicorner analysis results in exchange for speed. Such accuracy loss can be significant in more advanced nodes due to highly non-linear timing effects and more complex timing models. Since the circuit structure does not change across different corners, a large amount of data parallelism remains unused.

C. GPU-Accelerated STA Challenges

Hybrid compute systems with heterogeneous computing resources like CPUs and GPUs are becoming increasingly prevalent. Contrary to a CPU consisting of a few high-performance big cores, a GPU is made up of thousands of tiny cores arranged into streaming multiprocessors. It attempts to achieve a high throughput through extensive parallelization while minimizing threading costs. For instance, the RC timing computation for different nets can be done independently because the RC delay and slew parameters can be isolated [18]. Furthermore, the GPU's compute capability can also be used to sort out dependent tasks by computing a topological order in parallel. We highlight below three challenges for speeding up these STA tasks.

1) *Irregular Computational Patterns:* Nearly all STA tasks contain irregular computation patterns, including recursive procedures, dynamic data structures, and graph traversal.
2) *Frequent Memory Access:* Even though the RC delay computation for different nets is independent, each of them requires randomly accessing GBs of memory on million-gate designs, due to detailed parasitics and various local RC tree structures.
3) *Large Multicorner Memory Footprint:* In multicorner analysis, memory access problems are worsened by an additional 10×–100×, due to memory footprint proportional to the number of corners. This gives even more pressure on memory and cache management.

The above challenges require very strategic data structure models and decomposition algorithms to obtain a reasonable runtime speed-up.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

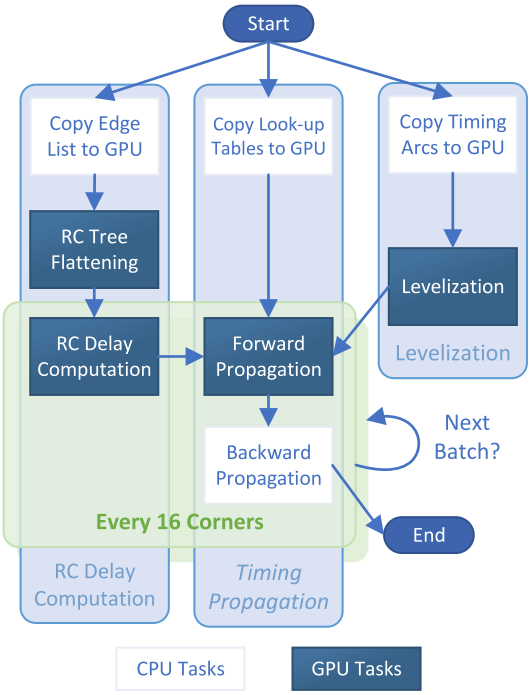


Fig. 4. Overall taskflow of our GPU-accelerated multicorner STA engine. The green frame captures the tasks running in parallel for a batch of corners.

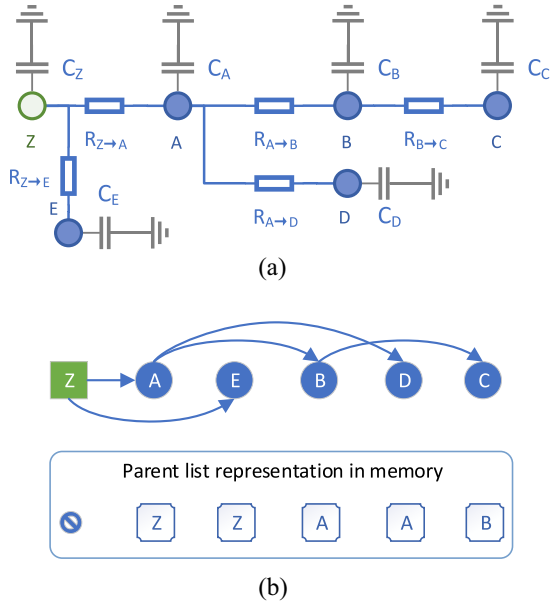


Fig. 5. Example of a net RC tree model with parasitics. (a) Edge resistance and node capacitance on parasitic RC tree. (b) BFS order of nodes is represented as a 1-D flattened array with a list of parent indices.

III. Algorithm

In this section, we present our GPU acceleration algorithm details. Our overall taskflow is presented in Fig. 4, where each arrow indicates a task dependency. There are three basic steps in our taskflow: 1) *RC delay computation*; 2) *timing propagation*; and 3) *levelization*. The timing propagation step is further decomposed to *forward propagation* and *backward propagation*. In Fig. 4, dark color indicates GPU-accelerated steps, including *RC delay computation*, *RC tree flattening*, *forward propagation*, and *levelization*. Because *backward propagation* has a much lower workload and nearly negligible runtime, we leave it on CPU. We focus on using GPU to address the major scalability issues and runtime bottlenecks shown in Fig. 2.

A. RC Delay Computation

A majority of STA runtime is taken by the RC delay computation step [18]. We begin by analyzing the model and the equations for calculating the delay and slew parameters through an RC network. Then we demonstrate how to build GPU-friendly data structures and algorithms. We adopt a variant of the Elmore delay model [18] to approximate interconnect delay, which had been widely adopted by many different STA engines [3], [8], [11]. We approximate the interconnect delay based on an Elmore delay model variant [18] that had been implemented by many STA engines [3], [8], [11]. As illustrated in Fig. 5, our goal is to calculate the impulse and delay between the source pin (*Port*) and each sink pin (*Taps*).

CPU Implementation [3]: Dynamic programming (DP) is a standard approach to implementing RC delay calculation. There are four stages in this algorithm.

1) Compute the pin load (i.e., the lumped capacitance) for each node u, denoted as load_u

$$\begin{aligned}\text{load}_A &= \text{Cap}_A + \text{Cap}_B + \text{Cap}_C + \text{Cap}_D \\ &= \text{Cap}_A + \text{load}_B + \text{load}_D.\end{aligned} \tag{1}$$

2) Compute the delay between *Port* and u, denoted as delay_u

$$\text{delay}_u = \sum_{v \in \text{nodes}} \text{Cap}_v R_{\text{Port}\to\text{LCA}(u,v)} \tag{2}$$

where LCA denotes lowest common ancestor

$$\begin{aligned}\text{delay}_B &= R_{Z\to A}\text{Cap}_A + R_{Z\to A}\text{Cap}_D \\ &\quad + (R_{Z\to A} + R_{A\to B})\text{Cap}_B \\ &\quad + (R_{Z\to A} + R_{A\to B})\text{Cap}_C \\ &= \text{delay}_A + R_{A\to B}\text{load}_B.\end{aligned} \tag{3}$$

3) Calculate the sum of capacitance and delay products in subtrees of u, denoted as ldelay_u, similar to step 1.
4) Calculate the beta and impulse parameters between the source *Port* and the sinks u, based on the ldelay of nodes, similar to step 2.

On CPU implementations, each parameter is typically computed using several passes of RC tree traversals. This yields a linear runtime complexity proportional to the tree size, although it may not be GPU-efficient due to its irregular recursions. Therefore, in our GPU implementation, we choose to use breadth-first search (BFS) traversals. In our BFS implementation instead, we precompute once a node order for each RC tree. This order ensures that every parent node is before any of its children. In other words, a tree edge $u \to v$ makes u appear before v in the BFS order, as illustrated in Fig. 5. The BFS order represents the structure of an RC tree concisely and efficiently for GPU execution. All we need to do is to visit the

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Flatten RC Trees

Input: N as #nets, (M, E) as (#nodes, #es) in all nets
Input: $roots[0..N-1]$, the root indices of each net
Input: $es[0..E-1]$, the undirected edge $\{(a, b)\}$
Input: $ndstart[0..N]$, the offsets of each net in node arrays, with $ndstart[N] = M$
Input: $estart[0..N]$, the offsets of each net in edge arrays, with $estart[N] = E$
Input: $dis[0..M] = \infty$, $cnts[0..M] = 0$
Output: $order[0..M-1]$, the BFS order for each net

```
/* Process one net w/ blockDim.x threads */
1  netID = blockIdx.x;                          ▷ gridDim.x = #nets
2  threadID = threadIdx.x;                      ▷ blockDim.x = 64
3  nst = ndstart[netID];                        ▷ start node offset
4  nend = ndstart[netID + 1];                   ▷ end node offset
5  est = estart[netID];                         ▷ start edge offset
6  eend = estart[netID + 1];                    ▷ end edge offset
7  dis[nst + roots[netID]] = 0;
8  for d = 0, 1, 2,..., (nend − nst) do
9      for i = est + threadID to eend step blockDim.x do
10         (a, b) = edgelist[i];
11         if dis[a] == d and dis[b] >d + 1 then
12             dis[b] = d + 1;
13             atomicAdd(cnts[d], 1);
14         end
15         else if dis[b] == d and dis[a] >d + 1 then
16             dis[a] = d + 1;
17             atomicAdd(cnts[d], 1);
18         end
19     end
20     __syncthreads();
21     break when cnts[d] == 0;
22 end
23 countingSort(dis, cnts, order, threadID);
```

Algorithm 2: Compute RC Delay for Corner Batches

Input: N as #nets, M as #nodes in all nets, BC as the batch size of multi-corner settings
Input: $st[0..N]$, the offsets of each net in arrays of nodes
Input: $parent[0..M-1]$, the index of parent of every nodes
Input: $resp[0..M-1]$, the resistance between nodes and their parent
Input: $cap[0..4M-1][BC]$, the capacitance of nodes, each in 4 different combinations
Output: $load[0..4M-1][BC]$, $delay[0..4M-1][BC]$, $impulse[0..4M-1][BC]$: arrays of results of load, delay and impulse, respectively

```
1  net = blockIdx.x × blockDim.x + threadIdx.x;
2  cond = threadIdx.y;
3  corner = threadIdx.z;
4  if net ≥ N then return;
5  offsetL = st[net];                           ▷ node offset start
6  offsetR = st[net + 1];                       ▷ node offset end
7  Initialize load, delay, ldelay to zero;
8  Initialize β = 0 as an auxiliary array;
9  for j = offsetR − 1 down to offsetL do
10     load[4j + cond][corner] += cap[4j + cond][corner];
11     load[4parent[j] + cond][corner] +=
         load[4j + cond][corner];
12 end
13 for j = offsetL + 1 to offsetR − 1 do
14     t = load[4j + cond][corner] × resp[j];
15     delay[4j + cond][corner] =
         delay[4parent[j] + cond][corner] + t;
16 end
17 for j = offsetR − 1 down to offsetL do
18     ldelay[4j + cond][corner] +=
         cap[4j + cond][corner] × delay[4j + cond][corner];
19     ldelay[4parent[j] + cond][corner] +=
         load[4j + cond][corner];
20 end
21 for j = offsetL + 1 to offsetR − 1 do
22     t' = ldelay[4j + cond][corner] × resp[j][corner];
23     β[4j + cond][corner] = β[4parent[j] + cond][corner] + t';
24     impulse[4j + cond][corner] =
         2β[4j + cond][corner] − delay[4j + cond][corner]^2;
25 end
```

tree nodes forwardly or backwardly via the ordered sequence, according to the DP update directions.

Algorithm 1 is our algorithm for GPU-accelerated RC tree flattening. It computes the node BFS order of one net using an input edge list in two stages: 1) computing distances to root for each node and 2) sorting nodes by their root distances. The time complexity of the first step is $\mathcal{O}(n^2)$, where n denotes the net size. Because of the limited net size (usually less than a few hundred), such an $\mathcal{O}(n^2)$ algorithm is efficient enough and can be even better parallelized on GPU due to its simplicity. A block of 64 threads is launched for each net to process its edge list. The edge list would be traversed multiple times to compute the order. In each iteration (lines 9–19), we obtain a new batch of nodes with the same root distance. Finally, we sort all nodes by their root distance using a GPU parallel counting sort with $\mathcal{O}(n)$ time complexity.

Based on the computed BFS order, the pseudocode for our RC computation GPU kernel is shown in Algorithm 2. We launch one kernel thread for each unique combination of Early/Late, Rise/Fall, net index, and corner index in a corner batch. The details of multicorner parallelism are introduced later in Section III-E. First, the *netID* and *condID* are computed on lines 1–4. We compute the net data offsets in parameter arrays on lines 5 and 6 and fill the output arrays with initial zero values on lines 7 and 8. After the initialization, we traverse and calculate the RC parameters load (lines 9–12), delay (lines 13–16), ldelay (lines 17–20), beta and impulse (lines 21–25).

Our algorithm works on our optimized RC tree data structure on GPU memory, where we store parent indices in the parent array for all RC tree nodes [Fig. 5(b)]. This concise representation of parent–child relations on GPU ensures a balanced workload during different DP update passes. We take the recursive equation of load as an example

$$\text{load}_u = \text{cap}_u + \sum_{v \in \{\text{children of } u\}} \text{load}_v \tag{4}$$

as also illustrated in Fig. 6(a). Because we only keep parent indices instead of a large adjacency list, we cannot enumerate all children of the node u and compute the sum as the equation requires. Instead, we equivalently regard the load_u as running sums. Algorithm 2 amend the running sum at u on all children of u (line 11), progressively arriving at the final result. Because the children of u appear after u in the BFS order, and because we scan the BFS order from backward, we would already

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

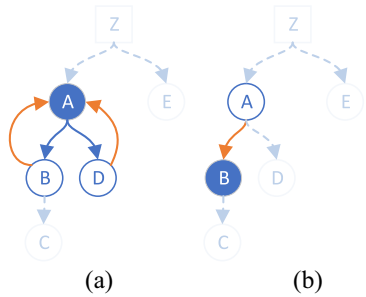


Fig. 6. Two different DP directions. (a) Upward recursive update, where the value of children needs to be computed before the value of parents. (b) Downward recursive update, where the parent values are computed before children.

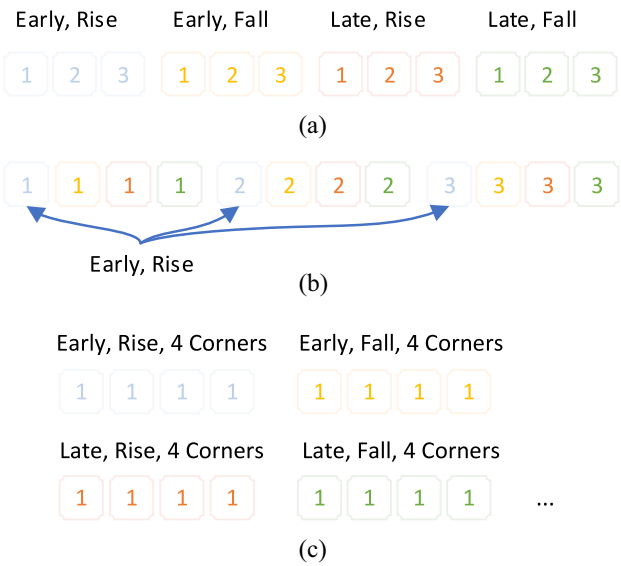


Fig. 7. Memory arrangement for Early/Late and Rise/Fall cases. (a) Independent access. (b) Interleaved access. (c) Interleaved access with multicorner optimization.

have processed all children of u before encountering u in the sequence.

Another example is the recursive equation of delay, which has a different direction

$$\text{delay}_v = \text{delay}_u + \text{pres}_v \times \text{load}_v \tag{5}$$

as shown in Fig. 6(b). Here, u denotes the parent of v. This equation is straightforward to implement using our array of parent indices and a forward BFS order scan (lines 14 and 15). The updates of ldelay and beta share similar patterns with load and delay and can be done analogously.

The bottleneck of RC computation is irregular global memory access with a large number of nets and independent analysis combinations. To address this, we design a data structure that is friendly for global memory access. We optimize memory bandwidth usage by interleaving the memory for the 4 Early/Late Rise/Fall combinations, instead of arranging them separately [Fig. 7(a) and (b)]. For multicorner analysis, we interleave different corners by assigning the corner dimension as the innermost array indices. This creates more memory coalescing capability as shown in Fig. 7(c). This arrangement

Algorithm 3: Levelization

Input: *nodes*, the set of graph nodes
Data: the current in-degree *in* and the adjacency list *out*
Output: a node level list

1 $F \leftarrow \{f \in nodes : in_f = 0\}$;
2 **while** *F is not empty* **do**
3 output F;
4 $F' \leftarrow \{\}$;
5 Call `advanceFrontiers` on F and get F';
6 $F \leftarrow F'$;
7 **end**
8 **Kernel Function** `advanceFrontier`**:**
Input: the old frontier F
Data: the adjacency list *out*, in-degree array *in*
Output: the new frontier F'
9 *nodeID* $\leftarrow$ **blockIdx**.x $\times$ **blockDim**.x $+$ **threadIdx**.x;
10 **if** *nodeID* $\geq$ size(F) **then return**;
11 **for** *v in out*[*nodeID*] **do**
12 *oldvalue* $\leftarrow$ `atomicAdd(`*in*[*v*], *-1*`)`;
13 **if** *oldvalue* $=$ *1* **then**
14 Add v to F';
15 **end**
16 **end**
17 **return** G;

ensures that adjacent 4 threads emit adjacent memory requests, which corresponds to the index equation $4i + \text{cond}$ for the ith node and the condth combination in Algorithm 2.

B. Levelization

Levelization is an STA step that constructs level-by-level task dependencies for timing propagation tasks [3]. It takes up nearly 40% of the full timing runtime (shown in Fig. 2). The inefficiency is caused by its single-threaded nature. Existing STA engines, including commercial tools like [11], perform a single-threaded DFS or BFS on the circuit logic to construct a level list and guide the parallelization of node tasks. This data structure is very time-consuming to maintain. As a result, we present a levelization algorithm with GPU acceleration.

Algorithm 3 shows our GPU-accelerated levelization process. Our key idea is to keep a node set F at the current level, called *frontiers*. Nodes that have no input edges (i.e., circuit primary input pins) become the initial frontiers (line 1). The algorithm repeats through lines 3–6 until we have processed all nodes. On each iteration, a GPU kernel `advanceFrontiers` is invoked to find the next frontiers based on current ones in parallel.

The `advanceFrontiers` procedure accepts a list of current frontiers and launches one thread to process a single frontier. The output edges of frontiers (lines 11–16) are enumerated. We decrement the in-degree of a node v by one for each output edge pointing to it. We push v to the next set of frontiers once its in-degree drops to zero after a decrement.

In this algorithm, the edge explorations of different frontiers are performed simultaneously, while the output edges from one frontier are processed one by one. The workload among GPU threads is proportional to the out-degree of nodes, which can be very imbalanced. We have adopted a *reverse* technique to tackle this problem by observing that the in-degrees of nodes

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

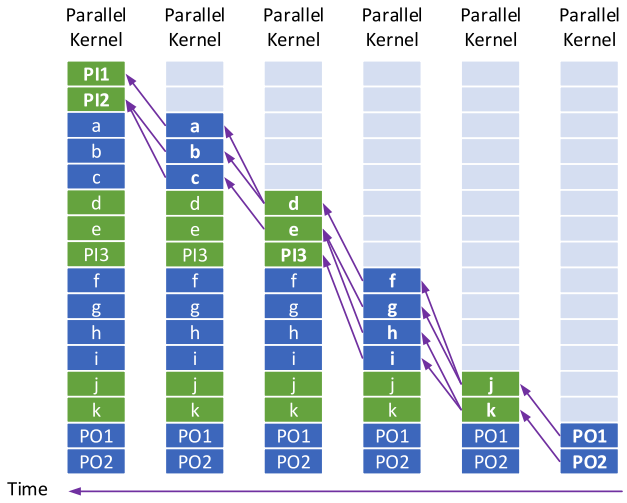


Fig. 8. Resulting level list for the circuit graph in Fig. 1, as well as the levelization process on GPU. Node names in bold indicate frontiers at the current iteration.

are generally smaller and much more balanced. For example, in netcard [18] with 1.5M of gates, the maximum out-degree and in-degree are 260 and 8, respectively. We reverse the edge directions before running the levelization on the graph, which gives higher parallelism during the edge exploration. After the levelization, we can retrieve the level orders of the original graph by reversing back the level orders, as illustrated in Fig. 8. The level list of large designs typically gives thousands of independent node tasks in each level, leading to enough parallelism for propagation.

C. Timing Propagation and LUT

According to the runtime decomposition in Fig. 2, the timing propagation step is efficient on CPU because of the small LUT size. Despite this, we managed to obtain a modest speed-up, especially for million-gate circuit designs, by migrating it to GPU. In the NLDM model, the delay and slew for cell arcs are modeled by a piecewise linear 2-D function with input slew and output capacitance as its inputs. This function is characterized by around 7×7 sample points obtained from circuit simulations and queried by bilinear interpolation.

We present our GPU-accelerated LUT table lookup algorithm in Algorithm 4. We calculate 2-D bilinear interpolations using three 1-D linear interpolations of sample points (lines 18–20). Given a single x, each 1-D linear interpolation finds the y of a piecewise linear polyline at x. When the given x exceeds the range of sample points, an extrapolation is performed instead of interpolation, which introduces a branch divergence on GPUs. To this end, we generalize the process to cover both extrapolation and interpolation under the same code, as illustrated in Fig. 9. The idea is to locate the line segment (or half-line) where x locates and then use a unified equation to solve y. We deal with the cases where LUT degenerated to a row, a column, or a single value, by setting $i' = i$ in these cases. A linear search is performed to find these indices because of the small size of LUTs.

Algorithm 4: Multicorner LUT Interpolation

```
   /* Input: line (x1, y1)--(x2, y2)                          */
   /* Input: the x value queried                              */
1  Function interpolate(x1, x2, y1, y2, x):
2      if x1 = x2 then return y1;
3      else return d1 + (d2 − d1) (x − x1)/(x2 − x1);
4  end
   /* Input: n × m look-up table with corner
      batch BC                                                */
   /* Input: the point queried (x, y) and the
      corner index corner                                     */
5  Function lut_lookup(n, m, X, Y, mat, x, y, corner):
6      i' ← 0;
7      i ← min(1, n − 1);
8      while i + 1 < n and X[i] ≤ x do
9          i' ← i;
10         i ← i + 1;
11     end
12     j' ← 0;
13     j ← min(1, m − 1);
14     while j + 1 < m and Y[j] ≤ y do
15         j' ← j;
16         j ← j + 1;
17     end
18     r_i' ← interpolate(Y[j'], Y[j], mat[i', j'][corner],
         mat[i', j][corner]);
19     r_i ← interpolate(Y[j'], Y[j], mat[i, j'][corner],
         mat[i, j][corner]);
20     r ← interpolate(X[i'], X[i], r_i', r_i);
21     return r;
22 end
```

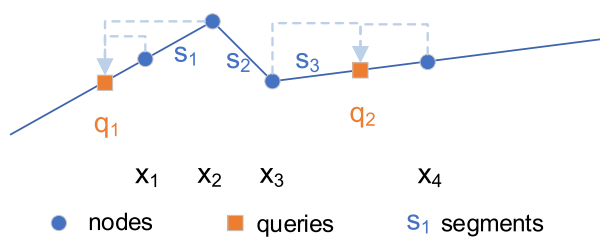


Fig. 9. Example of 1-D LUT query. This 1-D LUT is essentially a piecewise linear function with three segments s_1, s_2, and s_3. There are two queries q_1 (that hits s_1) and q_2 (that hits s_3). We get the result by evaluating the queried x-value on the specific segment, regardless of interpolation (like q_2) or extrapolation (like q_1).

D. Device-to-Device Data Bridge Between STA Steps

During the STA process, consecutive STA steps need to exchange intermediate timing analysis data. For example, the RC delay step computes net delay, capacitive load, and impulse, which are used in propagation to compute the signal AT. However, such communication is difficult to handle due to the data-structural difference between flattened RC trees and levelized arc tables. In STA engines like OpenTimer [3], a CPU is used to "translate" the timing data between different structures. In the multicorner case, this is no longer scalable due to the data size proportional to the number of corners.

To solve the above problem, we present Algorithm 5 as our device-to-device data transfer algorithm that bridges the gap between RC delay computation and timing forward propagation. The algorithm makes use of both CPU and GPU.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: Device-to-Device Data Transfer

```
   /* CPU code                                                    */
1  arcid2flatpos ← [];
2  for every net i do
3      r ← driver pin of net i;
4      L, R ← the range of net i's flattened RC storage;
5      for every sink pin p in net i do
6          t ← the index of arc r → p;
7          t' ← the position of p in [L, R];
8          arcid2flatpos[t] ← t';
9      end
10 end
11 copy arcid2flatpos to GPU;
   /* GPU code                                                    */
12 delay, impulse ← the GPU RC delay/impulse array;
13 arcdelay, arcimpulse ← the GPU flattened arc table;
14 arcid ← blockIdx.x × blockDim.x + threadIdx.x;
15 elrf ← threadIdx.y;
16 corner ← threadIdx.z;
17 arcdelay[arcid, elrf, corner] ← delay[arcid2flatpos[arcid],
     elrf, corner];
18 arcimpulse[arcid, elrf, corner] ← impulse[arcid2flatpos[arcid],
     elrf, corner];
```

On CPU, the algorithm preprocesses a mapping of memory offsets between flattened RC trees and the flattened arc table (lines 1–10). The mapping itself is small in size because it is set up only once for every single net arc, regardless of the number of corners and signal rise/fall conditions. After the mapping is ready, the algorithm copies it to GPU (line 11), where it is used to move groups of timing data to their destined locations (lines 12–18).

E. Multicorner GPU Parallelization

For each PVT condition under multicorner STA, the STA engine computes the circuit delay and slack using a unique cell library with its own set of pin loads and LUTs. Parallelization between corners can exceed tens or even hundreds, with the following properties.

1) While pin loads and LUTs are remodeled, the circuit topology remains unchanged across different corners. The topology-related computation (including RC tree flattening in Section III-A and levelization in Section III-B) can thus be cached and reused.
2) The computation across different corners have similar patterns and almost no branch divergence. This leads to efficient GPU-friendly single-instruction–multiple-thread (SIMT) behavior.

Fully utilizing these properties, we develop a GPU-accelerated multicorner STA flow. Storing all intermediate results for hundreds of corners is impractical on the limited GPU memory. Moreover, the thread block size proportional to the number of concurrent corners poses inefficient restrictions on GPU thread block scheduling. As such, we split the PVT corners into equal-sized batches with *BC* corners each, and compute the batches iteratively. In order to maximize data parallelism and SIMT performance, we put the corner iteration into the most inner loop, i.e., the last index of the 3-D CUDA thread group. Meanwhile, we also arrange the memory structure similar to Fig. 7 so that memory requests from consecutive corners are interleaved and thus coalesced.

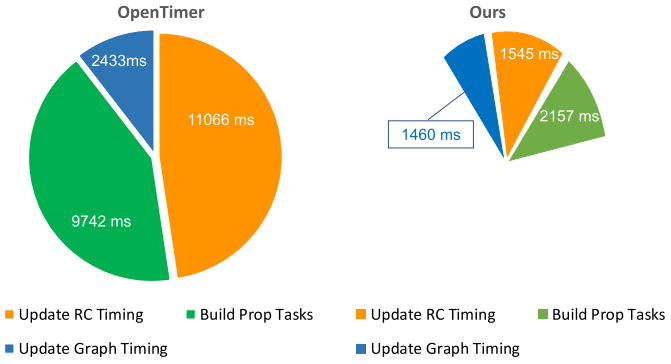


Fig. 10. Runtime breakdown of the circuit leon2 (21M nodes).

IV. Experimental Results

We implemented our GPU-accelerated STA algorithm on top of OpenTimer [3] and evaluated the results using TAU15 contest benchmarks [18]. We resynthesized all the benchmark netlists under an industrial 14-nm technology. This gives realistic multicorner cell libraries under different operating conditions. We do not compare with commercial tools (e.g., PrimeTime and OpenSTA) because they do not support GPU. Also, such a comparison may not be fair because of different application scopes. All experiments are undertaken on an Ubuntu Linux machine with 40 CPU cores at 2.10 GHz, 512-GB RAM, and 1 Nvidia A40 GPU. We configured the kernel execution with about 128 threads per block for all GPU kernels. Our algorithms are implemented using the parallel task programming framework, Taskflow [5], [27] to schedule CPU–GPU dependent tasks. We measured the *end-to-end* STA runtime including both GPU kernels and memory preparation operations.

A. Single-Corner Full Timing

Table I lists the benchmark statistics and the overall performance comparison between our approach and OpenTimer. We measure the runtime to complete one iteration of full-timing update on 15 benchmarks. The netlists of these benchmarks were used in TAU15 Contest to evaluate contestants' entries at a large scale. The gates of these netlists are remapped to a new cell library under 14-nm technology. We ran both OpenTimer and our algorithm using the maximum hardware concurrency of 16 CPUs and 1 GPU on our platform. Our runtime is faster than OpenTimer across all but the smallest benchmarks. The three largest speed-up values we observed are 4.05× on netcard (1.5M gates), 4.07× on leon2 (1.6M gates), and 3.51× on leon3mp (1.2M gates). The speed-up values become remarkable at large designs when generated STA graphs contain tens of millions of nodes and edges.

Fig. 10 shows the runtime breakdown of OpenTimer and our algorithm for notable items (>1000 ms) on the largest benchmark, leon2. OpenTimer spends 9742 ms to sort out the pin dependency, due to its unavoidable overhead on additional data structures and sequential nature. In our implementation,

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE COMPARISON BETWEEN OPENTIMER (16 CPUS) AND OUR GPU-ACCELERATED IMPLEMENTATION (1 GPU) TO COMPLETE ONE ITERATION OF FULL TIMING ON LARGE DESIGNS (>10K GATES) OF TAU 2015 CONTEST BENCHMARKS UNDER 14-NM TECHNOLOGY

Benchmark	# PIs	# POs	# Gates	# Nets	# Pins	# Nodes	# Edges	OpenTimer Runtime (16 CPUs)	Our Runtime [28] (16 CPUs 1 GPU)	
									Runtime	Speed-up
aes_core_14nm	260	129	22938	23199	66221	413058	499688	276 ms	283 ms	0.98×
vga_lcd_14nm	89	109	139529	139635	380730	1949332	2636815	1368 ms	659 ms	2.08×
vga_lcd_iccad_14nm	85	99	259067	259152	662179	3539206	4234464	2612 ms	951 ms	2.75×
b19_14nm	22	25	255278	255300	776320	4416480	5623578	3520 ms	1155 ms	3.05×
cordic_ispd_14nm	34	64	45359	45393	127993	730590	910649	508 ms	369 ms	1.38×
des_perf_ispd_14nm	234	140	138878	139112	371587	2095933	2473864	1679 ms	649 ms	2.59×
edit_dist_ispd_14nm	2562	12	147650	150212	416609	2555873	3562491	2056 ms	799 ms	2.57×
fft_ispd_14nm	1026	1984	38158	39184	116139	631491	868498	457 ms	353 ms	1.29×
leon2_14nm	615	85	1616369	1616984	4178874	22450936	28114268	23928 ms	5879 ms	4.07×
leon3mp_14nm	254	79	1247725	1247979	3267993	17647115	22807349	18174 ms	5174 ms	3.51×
netcard_14nm	1836	10	1496719	1498555	3901343	21023425	25014009	21320 ms	5259 ms	4.05×
mgc_edit_dist_14nm	2562	12	161692	164254	444693	2431266	3355118	1913 ms	793 ms	2.41×
mgc_matrix_mult_14nm	3202	1600	171282	174484	489670	2710343	3415291	1906 ms	798 ms	2.39×
pci_bridge32_14nm	160	201	40790	40950	108172	577083	696170	404 ms	318 ms	1.27×
tip_master_14nm	778	857	37715	38493	95524	533690	602224	341 ms	338 ms	1.01×

 # PIs: number of primary inputs **# POs**: number of primary outputs **# Gates**: number of gates **# Nets**: number of nets
 # Pins: number of pins **# Nodes**: number of nodes in the STA graph **# Edges**: number of edges in the STA graph

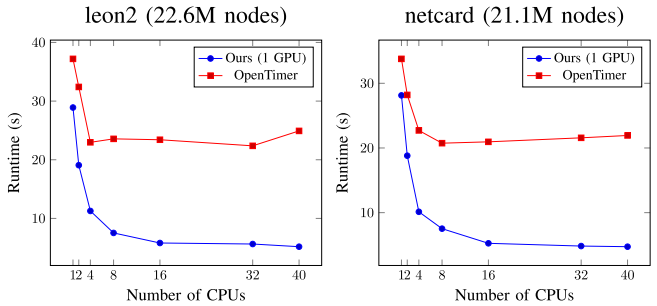


Fig. 11. Runtime values at different numbers of CPUs. Our runtime under 2 CPUs and 1 GPU is faster than OpenTimer of 16–40 CPUs.

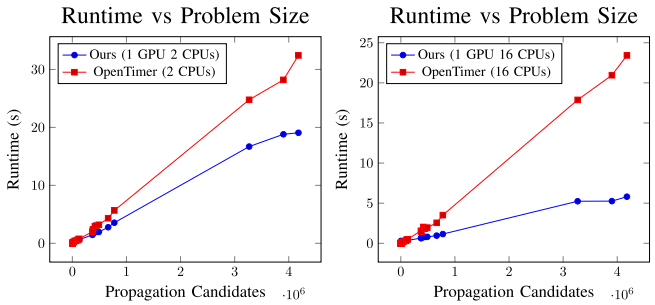


Fig. 12. Runtime values at different problem sizes. Beyond about 60K propagation candidates, our runtime is always faster than OpenTimer at any CPU numbers.

we use GPU to levelize the graph and run multiple tasks (e.g., update RC timing) in a single batch. We do not need as many tasks as OpenTimer but a single kernel to establish the topological dependency, which leads to just 2157-ms runtime. We observe a large amount of runtime reduction from updating RC timing. It takes 11 066 ms for OpenTimer to finish RC timing whereas we reach the goal by 7.16× faster. Our runtime for updating the graph timing is a bit faster (1460 ms versus 2433 ms), due to our GPU-based LUT interpolation.

Fig. 11 draws the runtime scalability versus increasing numbers of CPUs on the two largest designs, leon2 and netcard. Increasing the number of CPUs can speed up our overlapped CPU–GPU tasks with faster data transforms. We observe both methods scale up to 10 CPUs. Regardless of CPU numbers, our runtime is always faster than OpenTimer, and there exists a remarkable gap. The largest speed-up occurs at 40 CPUs, where ours is faster than OpenTimer by 4.81× on leon2. These results clearly demonstrate the strength of our approach that unleashes the computing power of GPUs beyond the limitation of CPU-based parallelism.

B. Single-Corner Incremental Timing

The success of GPU acceleration relies on a large enough data size and computation, which is abundant in the case of full timing updates on STA graph. During incremental timing, computation varies and may scope to a small local region or the entire timing landscape. Pins in this region are called *propagation candidates* which are the union of fan-in and fan-out cones spanned by frontier pins in incremental timing [3]. Considering the distinct performance characteristics between CPU and GPU, the most effective approach to incremental timing is a mixed strategy. When the number of propagation candidates is large, we use GPU; or we fall back to the existing CPU version of OpenTimer when propagation candidates are scarce.

Figs. 12–14 compare runtime at different sizes of problem candidates, nets, and gates, respectively, between our GPU algorithm and OpenTimer under different CPU concurrency. For problem size smaller than 10K, we run slower than OpenTimer but the runtime difference is negligible (< 80 ms). Beyond the threshold of 67K propagation candidates, our runtime is always faster than OpenTimer. The performance margin becomes bigger as we increase the problem size. In terms of the number of nets, the threshold is about 40K nets. We observe little benefit at small net counts due to the data and kernel overheads, but we are consistently faster at larger net counts. The threshold of gate numbers is roughly 45K, which corresponds to 360K LUTs. As LUT interpolation is less

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISON BETWEEN OUR SINGLE-CORNER STA ENGINE [28] AND OUR MULTICORNER STA ENGINE UNDER DIFFERENT MULTICORNER BATCH SIZE $BC = 2, 4, 8, 12, 16$ TO COMPLETE A 128-CORNER TIMING ANALYSIS ON GIVEN DESIGNS

Benchmark	One by One Runtime	2-way Parallel		4-way Parallel		8-way Parallel		12-way Parallel		16-way Parallel	
		Runtime	Speed-up	Runtime	Speed-up	Runtime	Speed-up	Runtime	Speed-up	Runtime	Speed-up
aes_core_14nm	36198	9984	3.63×	6005	6.03×	4896	7.39×	4693	7.71×	4661	7.77×
vga_lcd_14nm	84369	28459	2.96×	18635	4.53×	13611	6.20×	13053	6.46×	11192	7.54×
vga_lcd_iccad_14nm	121711	43157	2.82×	28384	4.29×	20672	5.89×	18861	6.45×	18539	6.57×
b19_14nm	147849	55573	2.66×	36032	4.10×	27067	5.46×	25087	5.89×	22251	6.64×
cordic_ispd_14nm	47241	11968	3.95×	8352	5.66×	6283	7.52×	6208	7.61×	5752	8.21×
des_perf_ispd_14nm	83132	25067	3.32×	16277	5.11×	11632	7.15×	11158	7.45×	9659	8.61×
edit_dist_ispd_14nm	102255	30784	3.32×	21216	4.82×	15472	6.61×	14733	6.94×	13275	7.70×
fft_ispd_14nm	45244	10837	4.17×	7829	5.78×	6997	6.47×	6057	7.47×	6091	7.43×
leon2_14nm	752512	306496	2.46×	195424	3.85×	152859	4.92×	139099	5.41×	133053	5.66×
leon3mp_14nm	662263	250795	2.64×	171936	3.85×	122939	5.39×	110447	6.00×	105013	6.31×
netcard_14nm	673109	276992	2.43×	172576	3.90×	122229	5.51×	111485	6.04×	106096	6.34×
mgc_edit_dist_14nm	101547	34069	2.98×	20811	4.88×	15888	6.39×	14285	7.11×	13523	7.51×
mgc_matrix_mult_14nm	102153	32000	3.19×	22944	4.45×	15403	6.63×	14527	7.03×	13608	7.51×
pci_bridge32_14nm	40670	10133	4.01×	7275	5.59×	5589	7.28×	5163	7.88×	4939	8.23×
tip_master_14nm	43221	9643	4.48×	7499	5.76×	5477	7.89×	5265	8.21×	4861	8.89×

Runtime: The runtime for analyzing 128 corners in milliseconds.
One by One: Running our GPU-accelerated single corner analysis [28] for 128 times.
BC**-way Parallel**: Running our GPU-accelerated multi-corner analysis with batch size set to BC, for $128/BC$ times.

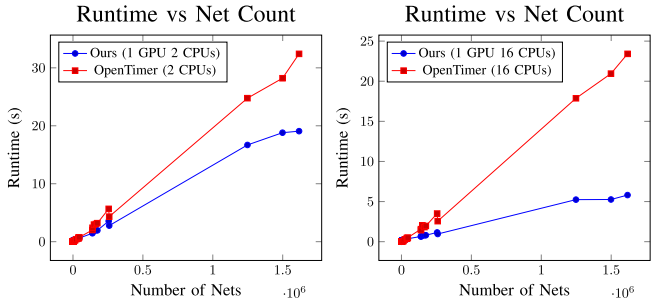


Fig. 13. Runtime values at different net counts. Beyond about 40K nets, our GPU-accelerated RC computation is always faster than OpenTimer, regardless of CPU numbers.

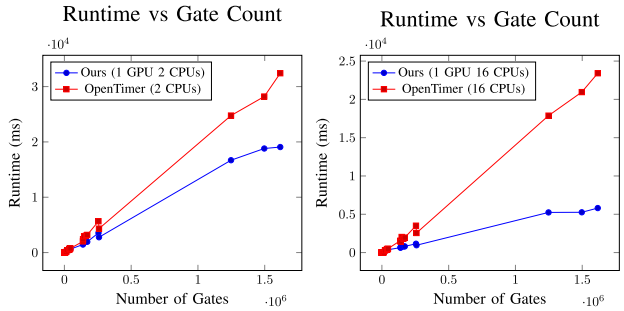


Fig. 14. Runtime values at different numbers of gates (∼LUT numbers). Beyond about 45K gates, our GPU-accelerated LUT interpolation becomes faster than OpenTimer.

data- and compute-intensive than other tasks, the performance margin is expected to become closer with increasing number of CPUs. To sum up, the performance benefits of our GPU-accelerated STA algorithm are remarkable when applications define large numbers of propagation candidates, for example, timing-driven placement and routing [29], [30].

C. Multicorner Analysis

In this section, we present our results on multicorner STA acceleration. Our industrial 14-nm technology includes

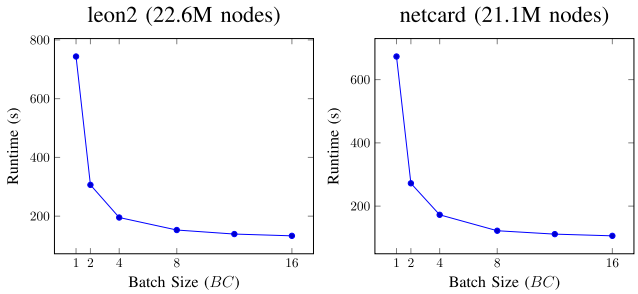


Fig. 15. Runtime values at different corner batch sizes (BC) for analyzing 128 corners. The data point at $BC = 1$ comes from our GPU-accelerated single-corner STA engine, which is our conference version [28]. Other data points are collected using our multicorner STA engine.

a diverse range of cell libraries under voltages ranging in [0.66, 0.99], temperatures ranging in [−40c, 125c], and three different process corners (ff, ss, and tt). We choose 128 corners from all available libraries for testing. Table II shows a detailed runtime comparison between our single-corner and multicorner analysis, with multicorner batch size set to $BC = 2, 4, 8, 12$, and 16. Despite both running on GPU, our multicorner algorithm outperforms our original single-corner algorithm by a large amount. On large designs like leon2, leon3mp, and netcard, we can achieve 3.85×–3.90× speed-up by computing 4 corners in parallel, compared to computing corners one by one. A larger batch size gives better performance. By computing 16 corners in parallel, the speed-up on leon2, leon3mp, and netcard is enlarged to 5.66×–6.34×. These results have proven the efficiency of our GPU-accelerated multicorner STA algorithm on exploring data parallelism across corners.

Fig. 15 visualizes the runtime with respect to BC on the two largest designs, leon2 and netcard. With our multicorner acceleration techniques, a drastic speed-up can be obtained compared to the state-of-the-art GPU-accelerated single-corner STA engine ($BC = 1$ in Fig. 15). A larger batch size leads to a better performance, which saturates at around $BC = 16$.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

Note that regarding data parallelism, a batch size of 16 already fulfills the half-warp SIMT dispatching scheme of the current CUDA architecture and can eliminate branch divergence completely.

Note that when compared with the original CPU-based OpenTimer, a speed-up ratio in Table II should be multiplied with the GPU acceleration speed-up in Table I, which is itself 3.51×–4.07×. This yields an overall speed-up of 22.14× –25.67× compared to running OpenTimer repeatedly for all corners. We also note that the speed-up ratio is larger for smaller designs. For example, on cordic_ispd, des_perf_ispd, and tip_master, the multicorner speed-up ratio is more than 8×. Such counterintuitive results may come from their smaller memory footprint, due to our extensive GPU memory usage proportional to the batch size *BC* used.

V. Conclusion

In this article, we have presented a new GPU-accelerated STA algorithm to go beyond the scalability of existing methods. We have developed GPU-efficient data structures and algorithms to speed up essential tasks, including levelization, delay computation, and timing propagation in updating an STA graph. We have leveraged task parallelism to describe dependent CPU–GPU tasks such that data processing and kernel computation are efficiently overlapped. We have scaled our GPU acceleration to the analysis of multiple PVT corners, which yields further runtime improvements. Compared to the state-of-the-art STA engine, OpenTimer, we achieved up to 4.07× speed-up on a large design of 1.6M gates and 1.6M nets using 1 GPU. By computing 16 corners in parallel, we achieved another 5.66× speed-up.

Our future work includes developing GPU-accelerated algorithms for different delay calculators, including current source cell delay models and reduced-order wire delay models. We also plan to incorporate GPU task parallelism using CUDA graph feature [31] to reduce the overhead of CUDA streams and enable multiple GPUs acceleration for other time-consuming STA tasks (e.g., path-based analysis [32], [33]).

References

[1] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*, 1st ed. New York, NY, USA: Springer, 2009. [Online]. Available: https://link.springer.com/book/10.1007/978-0-387-93820-2

[2] W. E. Donath and D. J. Hathaway, "Distributed static timing analysis," U.S. Patent 6 557 151, Apr. 29, 2003.

[3] T.-W. Huang, G. Guo, C.-X. Lin, and M. D. F. Wong, "OpenTimer v2: A new parallel incremental timing analysis engine," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 4, pp. 776–789, Apr. 2021.

[4] T.-W. Huang, C.-X. Lin, G. Guo, and M. D. F. Wong, "OpenTimer: A high-performance timing analysis tool," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2015, pp. 895–902.

[5] T.-W. Huang, C.-X. Lin, G. Guo, and M. D. F. Wong, "CPP-Taskflow: Fast task-based parallel programming using modern C++," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2019, pp. 974–983.

[6] T.-W. Huang, M. D. F. Wong, D. Sinha, K. Kalafala, and N. Venkateswaran, "A distributed timing analysis framework for large designs," in *Proc. 53rd ACM/IEEE Design Autom. Conf. (DAC)*, 2016, pp. 1–6.

[7] K. E. Murray and V. Betz, "Tatum: Parallel timing analysis for faster design cycles and improved optimization," in *Proc. Int. Conf. Field Program. Technol. (FPT)*, 2018, pp. 110–117.

[8] Y.-M. Yang, Y.-W. Chang, and I. H.-R. Jiang, "iTimerC: Common path pessimism removal using effective reduction methods," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2014, pp. 600–605.

[9] T.-W. Huang and M. D. F. Wong, "UI-timer 1.0: An ultrafast path-based timing analysis algorithm for CPPR," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 11, pp. 1862–1875, Nov. 2016.

[10] P.-Y. Lee, I. H.-R. Jiang, and T.-C. Chen, "FastPass: Fast timing path search for generalized timing exception handling," in *Proc. IEEE 23rd Asia South Pac. Design Autom. Conf. (ASP-DAC)*, 2018, pp. 172–177.

[11] "OpenSTA." Accessed: May 1, 2023. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenSTA

[12] A. B. Kahng, U. Mallappa, L. K. Saul, and S. Tong, "unobserved corner prediction: Reducing timing analysis effort for faster design convergence in advanced-node design," in *Proc. Design Autom. Test Europe Conf. Exhibit. (DATE)*, 2019, pp. 168–173.

[13] S. Onaissi, F. Taraporevala, J. Liu, and F. N. Najm, "A fast approach for static timing analysis covering all PVT corners," in *Proc. ACM 48th Design Autom. Conf. (DAC)*, 2011, p. 777.

[14] J.-J. Nian, S.-H. Tsai, and C.-Y. Huang, "A unified multi-corner multi-mode static timing analysis engine," in *Proc. IEEE 15th Asia South Pac. Design Autom. Conf. (ASP-DAC)*, 2010, pp. 669–674.

[15] S. Sripada and M. Palla, "A timing graph based approach to mode merging," in *Proc. ACM 52nd Annu. Design Autom. Conf.*, 2015, pp. 1–6.

[16] S. Onaissi and F. N. Najm, "A linear-time approach for static timing analysis covering all process corners," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1291–1304, Nov. 2008.

[17] L. G. e Silva, L. M. Silveira, and J. R. Phillips, "Efficient computation of the worst-delay corner," in *Proc. IEEE Design Autom. Test Europe Conf. Exhibit.*, 2007, pp. 1–6.

[18] J. Hu, G. Schaeffer, and V. Garg, "TAU 2015 contest on incremental timing analysis," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2015, pp. 882–889.

[19] J. Hu, D. Sinha, and I. Keller, "TAU 2014 contest on removing common path pessimism during timing analysis," in *Proc. Int. Symp. Phys. Design*, 2014, pp. 153–160.

[20] J. Hu et al., "TAU 2016 contest on macro modeling," in *Proc. ACM Int. Workshop Timing Issues Specification Synth. Digit. Syst.*, 2016, p. 9.

[21] T.-Y. Lai, T.-W. Huang, and M. D. Wong, "LibAbs: An efficient and accurate timing macro-modeling algorithm for large hierarchical designs," in *Proc. 54th Annu. Design Autom. Conf.*, 2017, pp. 1–6.

[22] P.-Y. Lee and I. H.-R. Jiang, "iTimerM: A compact and accurate timing macro model for efficient hierarchical timing analysis," *ACM Trans. Design Autom. Electron. Syst.*, vol. 23, no. 4, pp. 1–21, 2018.

[23] H. H.-W. Wang et al., "Casta: Cuda-accelerated static timing analysis for VLSI designs," in *Proc. IEEE 43rd Int. Conf. Parallel Process.*, 2014, pp. 192–200.

[24] K. Gulati and S. P. Khatri, "Accelerating statistical static timing analysis using graphics processing units," in *Proc. IEEE Asia South Pac. Design Autom. Conf.*, 2009, pp. 260–265.

[25] Y. Shen and J. Hu, "GPU acceleration for PCA-based statistical static timing analysis," in *Proc. 33rd IEEE Int. Conf. Comput. Design (ICCD)*, 2015, pp. 674–679.

[26] J. Cong et al., "Accelerating Monte Carlo based SSTA using FPGA," in *Proc. 18th Annu. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, 2010, pp. 111–114.

[27] T.-W. Huang, D.-L. Lin, C.-X. Lin, and Y. Lin, "Taskflow: A lightweight parallel and heterogeneous task graph computing system," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1303–1320, Jun. 2022.

[28] Z. Guo, T.-W. Huang, and Y. Lin, "GPU-accelerated static timing analysis," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2020, pp. 1–9.

[29] Z. Guo and Y. Lin, "Differentiable-timing-driven global placement," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2022, pp. 1315–1320.

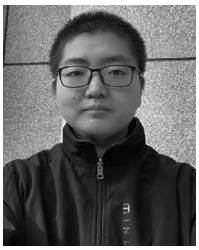
Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.

[30] P. Liao, D. Guo, Z. Guo, S. Liu, Y. Lin, and B. Yu, "DREAMPLACE 4.0: Timing-driven placement with momentum-based net weighting and Lagrangian-based refinement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, Jan. 30, 2023, doi: 10.1109/TCAD.2023.3240132.

[31] D.-L. Lin and T.-W. Huang, "Efficient GPU computation using task graph parallelism," in *Proc. Parallel Process. (Euro-Par)*, 2021, pp. 435–450.

[32] G. Guo, T.-W. Huang, Y. Lin, and M. D. F. Wong, "GPU-accelerated path-based timing analysis," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 721–726.

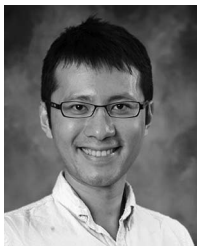
[33] G. Guo, T.-W. Huang, Y. Lin, Z. Guo, S. Yellapragada, and M. D. F. Wong, "A GPU-accelerated framework for path-based timing analysis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, early access, May 9, 2023, doi: 10.1109/TCAD.2023.3272274.



Zizheng Guo received the B.S. degree in computer science from Peking University, Beijing, China, in 2022, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits.

His research interests include data structures, algorithm design, and GPU acceleration for combinatorial optimization problems. He is currently working on static timing analysis and power analysis problems in VLSI CAD.

Mr. Guo is the First Place Winner of the 2022 ACM Student Research Competition Grand Finals.



Tsung-Wei Huang (Member, IEEE) received the B.S. and M.S. degrees from the Department of Computer Science, National Cheng Kung University (NCKU), Tainan, Taiwan, in 2010 and 2011, respectively, and the Ph.D. degree from the Electrical and Computer Engineering (ECE) Department, University of Illinois at Urbana–Champaign, Champaign, IL, USA, in 2017.

He was an Assistant Professor with the ECE Department, The University of Utah, Salt Lake City, UT, USA, from 2019 to 2023, and is currently an Assistant Professor with the ECE Department, University of Wisconsin at Madison, Madison, WI, USA. He has been building software systems for parallel computing and timing analysis.

Dr. Huang has received several prestigious awards to recognize his contributions, including the ACM SIGDA Outstanding Ph.D. Dissertation Award, the NSF CAREER Award, the Humboldt Research Fellowship Award, and the ACM SIGDA Outstanding New Faculty Award.

Yibo Lin (Member, IEEE) received the B.S. degree in microelectronics from Shanghai Jiao Tong University, Shanghai, China, in 2013, and the Ph.D. degree in electrical and computer engineering from the University of Texas at Austin, Austin, TX, USA, in 2018.

He is currently an Assistant Professor with the School of Integrated Circuits, Peking University, Beijing, China. His research interests include physical design, machine learning applications, and heterogeneous computing in VLSI CAD.

Dr. Lin is a recipient of the Best Paper Awards at premier EDA/CAD journals/conferences, such as TCAD, DAC, DATE, and ISPD.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:53:31 UTC from IEEE Xplore. Restrictions apply.